PACT Compute implementation and evaluation report

Technical report for the fifth semester project review

13 November 2026

Project purpose and scope

PACT Compute is a private GPU job control plane and experimental resource allocation platform. It connects a written service agreement to a reviewed policy, checks whether an available GPU can satisfy that policy, executes an approved image processing workload, and produces evidence describing what happened. Its website brings these activities together with a producer and consumer marketplace, research experiments, and explanations of decisions. The central contribution is the connection between agreement interpretation, resource allocation, actual execution, and subsequent verification within one application.

The practical problem is that access to a GPU alone does not establish whether a job should run. A user may require a minimum memory capacity, a completion deadline, a spending limit, an execution location, or energy reporting. A resource provider may have availability and pricing constraints. An accepted request must remain consistent with these terms while capacity changes and processes fail. PACT represents those terms explicitly and preserves the evidence needed to inspect a decision later.

The current implementation supports one exclusive GPU per live job and a predefined image embedding workload. It is not a general platform for running arbitrary uploaded programs or distributed training. Its research component evaluates scheduling and pricing on trace-derived and controlled data. These experiments are separate from the physical runner. This distinction is essential: a simulated allocation can support a research comparison, while a completed local job demonstrates execution on actual hardware.

The system is suitable for an academic demonstration of policy-aware GPU allocation and explainable evidence. Production readiness is a separate question involving durable storage, operational monitoring, backups, identity integration, deployment verification, and independent review. An HTTP health response is useful evidence that the API can reach its database, but it does not establish that a GPU runner is available or that every user workflow succeeds.

Architecture and technology choices

The frontend uses React and TypeScript. React organizes interactive screens into components, while TypeScript checks the shapes of values passed between them. The build process performs a TypeScript check and bundles the application with esbuild. The resulting HTML, JavaScript, and CSS are served by the backend. A separate frontend development server is therefore unnecessary for the normal demonstration. Browser routes provide access to agreements, jobs, the exchange, bookings, research, explanations, team membership, and operational views.

FastAPI provides the HTTP API. Pydantic models validate request data, including field lengths, supported choices, numeric bounds, and structured workload inputs. SQLAlchemy manages database access, with SQLite available for a local installation and PostgreSQL supported for a hosted installation. Alembic migrations track schema changes. The repository currently includes seven migration revisions, ending with incident records. Application code and schema must be kept compatible during deployment.

The API, research worker, and GPU runner are separate processes. The API authenticates requests, applies authorization, checks policies, and records state transitions. The worker claims research runs and performs replay computations outside the interactive request path. The runner polls the control plane, maintains heartbeats, accepts authorized work, executes the approved template, and returns results. Separating these responsibilities makes their failure modes visible: a healthy API can coexist with an unavailable runner or a stopped research worker.

The database stores organizations, accounts, memberships, sessions, policy revisions, runners, jobs, attempts, offers, quotes, bookings, artifacts, events, reviews, rate limits, and incidents. Large payloads are represented through artifact records and storage interfaces. An artifact can live on local disk or an S3-compatible backend. The database record and stored bytes must agree, which is why content hashing is checked when artifacts are written and retrieved.

The container entrypoint runs a supervisor for the API and research worker. If a required child process exits unexpectedly, the service stops instead of continuing to advertise a healthy partial installation. This detects process death, not every possible deadlock or hung computation. The native GPU runner remains outside the API container and requires an authorized NVIDIA host with its own runtime and enrollment credentials.

Organizations identities and permissions

An organization represents a workspace. Members act within an organization and receive capabilities according to their role. Producer and consumer signup creates separate workspaces. A producer supplies compute capacity, while a consumer requests execution under an approved agreement. Reviewers approve policies. Researchers inspect and run experiments, and operators manage the installation. The administrator is useful for a compact local demonstration but should not be used to imply that normal consumers can approve their own policies.

Membership invitations allow a team to bring another person into its workspace. The invited person redeems the invitation in a separate session. Workspace switching is checked against actual membership. Authorization is enforced by the API, rather than by merely hiding navigation links. A user who guesses an identifier should not gain access to another organization's jobs or private artifacts. Automated tests exercise these boundaries, including attempts to switch into foreign workspaces.

Passwords are salted and processed with PBKDF2 using SHA-256. Sessions have a finite lifetime, and the hosted configuration supports secure cookies over HTTPS. A runner uses its own scoped credential rather than a browser session. Signing credentials are separate from user passwords. These distinctions limit the impact of sharing the wrong credential and make it possible to enroll a runner without handing out an administrator login.

The exchange intentionally shares some information between organizations. The producer and successful consumer can inspect the same booking receipt, while the consumer's original job and input artifacts remain private. A losing consumer cannot read the winning booking merely because both requested the same offer. This controlled sharing is an important part of the implementation: collaboration requires a common transaction record without exposing all of a participant's workspace.

From service agreement to executable policy

A service level agreement describes the conditions under which a task should execute. PACT supports a bounded collection of rule types: minimum GPU memory, GPU count, GPU model, grid zone, relative or absolute deadlines, maximum deferral, maximum total charge, maximum hourly rate, carbon limits, lower-carbon preference, and reporting requirements. A bounded language is easier to validate and enforce consistently than unrestricted natural language.

The local extractor recognizes supported wording and converts it to typed values. Memory quantities are normalized to MiB. Durations become seconds. Credit amounts become integer hundredths of a credit. Absolute timestamps require a date and timezone and are normalized to UTC. Decimal arithmetic avoids treating financial limits as approximate binary floating point quantities. Strict limits, such as a charge being below a value, remain distinguishable from inclusive limits.

Each extracted rule retains a link to the original text through source offsets. This allows the reviewer to inspect the words supporting a decision. The example agreement contains six clauses: 4 GiB minimum memory, completion within 12 minutes, a four-credit charge limit, up to three minutes of deferral, a lower-carbon preference, and energy reporting. Extracting the example produces six source-linked rules, which can be inspected before approval.

Ambiguity is treated as a reason for review. For example, “Jobs must finish by 17:00” does not identify a date or timezone. The application should not silently decide what the author meant. Unsupported exceptions, unresolved statements, conflicting constraints, and invalid values can prevent approval. This is preferable to producing a confident policy whose semantics do not match the agreement.

Optional model-assisted extraction can use Groq or another configured provider. The model proposes candidates; source verification and privileged review still determine whether those candidates become an approved policy. The gateway applies a shared request budget, caching, cooldown, and single-flight behavior. If assistance is unavailable, local compilation and local guidance remain available. A language model is therefore an optional aid, not the authority that grants execution permission.

Approval turns a reviewed revision into usable policy. The system preserves the source and version associated with the decision. A policy change cannot safely rewrite the meaning of an already accepted booking. Quotes and workload specifications refer to relevant versions so that confirmation and dispatch can detect stale terms. This connects human review to the exact request that later executes.

Admission preview and placement

Admission asks whether a proposed job is feasible before reserving hardware. The website exposes a placement preview that evaluates the agreement against current inventory and availability. A policy can be perfectly valid yet infeasible on the available hardware. For example, a reviewed requirement for 32 GiB of memory should be rejected on a laptop GPU with approximately 6 GiB. This demonstrates a useful distinction between validating the agreement and satisfying it.

The preview checks the runner, capacity window, workload, timing, and pricing conditions. A rejected request identifies the relevant rule rather than presenting only a generic error. The user can record the rejected request to retain an audit trail. Counterfactual suggestions help explain how changing a constraint could affect feasibility. Accepting a suggestion creates a new draft; it does not silently modify the approved agreement or guarantee that every other requirement becomes feasible.

Live duration estimation currently uses a fixed template formula of 38 plus three times the number of images, in seconds. Four images therefore produce a planning estimate of 50 seconds. This is a reservation estimate, not a promise that the neural network computation itself lasts 50 seconds. Staging, model startup, execution, validation, and upload are distinct parts of the end-to-end path. The research runtime predictor is not silently substituted into live dispatch.

The shadow recommendation option records a placement recommendation without executing the workload or reserving GPU capacity. It is useful when explaining admission logic without consuming hardware. A shadow result must be described as a recommendation. It does not prove that the runner performed the task, that output was produced, or that a deadline was satisfied.

Physical GPU execution

The approved live workload accepts one to sixteen PNG, JPEG, or WebP images. The interface can also provide built-in sample images. Inputs are normalized into a manifest containing image data and hashes. The job specification records the template, model, input manifest, policy, hardware requirements, and runtime identity. Hashes bind the request to particular bytes, making accidental substitution or corruption detectable.

The model is MobileNetV3 Large with pretrained ImageNet weights. Its role here is feature extraction: each image becomes a 960-dimensional embedding. An embedding is a numerical representation of visual information that can support later similarity search or classification tasks. PACT does not train a new image classifier in this workflow. It also does not report a classification accuracy merely because an embedding vector was generated successfully.

The default execution path uses PyTorch to load weights and preprocess images, then executes the project's native kernels through the NVIDIA CUDA Driver API. Consequently, a CPU-labelled PyTorch wheel can still participate in an actual GPU workflow. Checking only whether PyTorch reports CUDA availability would be the wrong readiness test for this engine. The relevant evidence is a successful driver probe and execution with valid CUDA output.

Model preparation downloads or reuses the expected weight file and checks its upstream hash. The preparation path compares GPU behavior with a CPU reference. The runner checks input hashes before execution. Returned artifacts include an input-image hash for each output row, allowing the consumer to match vectors to their original images. Output validation checks dimensions and the expected format before the control plane accepts success.

The runner periodically reports heartbeats and execution state. Work is protected by leases and fencing values so that stale attempts cannot freely continue writing authoritative outcomes. If a callback cannot be confirmed, local journaling supports reconciliation. An expired lease does not automatically prove that the physical process has stopped. Capacity remains reserved while execution is uncertain until reconciliation or an explicit operator confirmation resolves the situation.

Cancellation and timeout handling are also part of the lifecycle. A request may be cancelled before workload start or while work is in progress. Time budgets include the remaining reservation window. The system records failed, expired, cancelled, rejected, recommended, and successful outcomes rather than treating every accepted submission as a success. This creates a more informative demonstration than a page that always displays a completed result.

Energy carbon and cost accounting

Energy and carbon are related but different measurements. The runner samples GPU board power through NVIDIA telemetry. Integrating power over time gives an estimate of GPU energy consumption. Power in watts multiplied by elapsed seconds is converted to kWh by dividing by 3,600,000. Sampling coverage matters: missing samples reduce confidence, and GPU board energy does not represent the entire laptop, cooling system, network, or datacenter.

Carbon estimation combines energy with electricity carbon intensity. In a simplified calculation, energy in kWh multiplied by intensity in grams of CO2 per kWh yields grams of CO2. A defensible live estimate needs the runner's actual grid zone, an appropriate forecast or observation, and a documented power bound. The implemented live connector supports Great Britain. A laptop in India must not be labelled as a British runner merely to populate the carbon display.

The research replay uses historical Great Britain carbon intensity from NESO. Its planning signal normally uses earlier observations as a forecast proxy. The oracle condition deliberately uses hindsight and should be

presented as an upper-information comparison. Neither condition establishes the emissions of the local laptop. Separating measured inputs, modeled assumptions, and hypothetical outcomes prevents attractive graphs from being mistaken for direct environmental measurements.

Compute credits represent internal usage and price constraints. Rates, maximum charges, accepted quote terms, and adjustments help explain the economic side of an execution. They are not bank payments or a complete accounting ledger. Real billing would require settlement rules, invoices, refunds, identity checks, and operational controls beyond the current demonstration.

GPU exchange and booking protocol

A producer publishes a capacity offer associated with an enrolled runner and an availability window. A consumer selects an approved agreement and requests a quote. The quote describes proposed terms and expires after a limited period. Requesting a quote does not permanently hold the slot. This allows the interface to show tentative options while reserving scarce capacity only when the consumer confirms.

Confirmation is the critical transition. The application rechecks offer and policy versions, timing, and capacity under database locking. If two consumers confirm competing quotes for the last slot, exactly one should obtain the reservation. The other receives a changed-slot response. A screenshot showing two quotes is not itself a double-booking error, because quotes are tentative. The invariant concerns confirmed overlapping reservations.

The booking links commercial terms to the associated execution. Both participants can inspect progress from accepted quote through lease, execution, and capacity release. Live updates use event delivery with reconnect support. Complete event history is important both for the interactive view and for later evidence exports. The implementation includes pagination and ordering safeguards intended to avoid losing events when histories become long or clients reconnect.

The shared receipt contains the transaction evidence that both parties are allowed to see. The producer and consumer receive matching receipts, while private job details remain scoped to the consumer. An HMAC signature protects the exported manifest against modification by parties without the signing secret. This establishes integrity relative to this installation's secret. It is not a public-key signature, public notarization, or proof against a compromised administrator holding that same secret.

Independent policy and protocol monitoring

Preventive checks and retrospective monitoring answer different questions. Admission asks whether execution appears permissible. Dispatch rechecks relevant conditions when work is about to begin. Monitoring reconstructs the observed sequence afterward and asks whether the policy and booking protocol were followed. Using recorded events for this reconstruction reduces reliance on a single final status flag.

Temporal properties concern ordering and relationships over time. A valid execution should follow the appropriate reservation and authorization events, use the right lease, and reach a coherent terminal state. Merely finding an event named “started” and another named “completed” is insufficient if they occurred in an invalid order or belonged to incompatible attempts. The protocol study explicitly compares a temporal monitor with an event-presence baseline.

Audit findings identify passes, failures, and unsupported or unknown conclusions where appropriate. The interface exposes execution stages and their supporting evidence. Operators can inspect incidents and record resolutions. A resolution should preserve the original problem and the reason for the intervention. It should not erase an inconvenient event from the history or make an uncertain execution appear automatically safe.

PostgreSQL event ordering currently serializes application writers to maintain a consistent sequence. This is a deliberate correctness trade-off for a small private installation. It limits the strength of scalability claims. All older writers must be drained before deploying the relevant ordering change, since a mixed population of writers cannot be assumed to share the same guarantee.

Dataset preparation and provenance

The primary imported snapshot contains 7,064 Alibaba GPU pods, 185 successful lifecycle proxies, and 1,344 half-hour carbon intervals. A download manifest records SHA-256 hashes for source files. Import verifies those hashes before writing normalized Parquet datasets and a data card. The card records the source identity and helps link later experiments to the dataset that actually produced them.

The successful lifecycle proxy spans scheduler-to-deletion observations and can differ from pure GPU compute duration. It includes only the usable successful observations selected by the implementation. Failed, incomplete, and censored workloads are not represented as ordinary successful runtimes. This selection limits what can be concluded about arbitrary production jobs or heavily failure-prone workloads.

Chronological splitting separates earlier observations used for fitting from later observations used for evaluation. The runtime study uses 102 training, 37 calibration, and 46 test proxies. This is more realistic than freely mixing future and past observations, although the sample remains small. A model that performs well on these proxies still needs evaluation on the actual workload classes and hardware expected in deployment.

Research runs retain seeds, dataset identifiers, offered prices, allocations, assumptions, and audit transitions. Reusing the same arrivals and offer tape across scheduler arms creates paired comparisons. A difference can then be attributed more directly to the scheduling condition rather than an accidental change in random demand. Reproducibility requires preserving both code identity and configuration, not only saving a chart.

Scheduling theory and experiments

Scheduling chooses which jobs run, where they run, and when they start under limited capacity. The Research Lab compares first come first served, earliest deadline first, policy-only allocation, carbon-aware allocation, and an unconstrained comparison. Additional offline baselines include backfilling and deadline scheduling with aging. Each baseline makes a different trade-off between simplicity, feasibility, fairness, waiting time, and environmental objectives.

First come first served prioritizes arrival order. Earliest deadline first prioritizes urgency. Policy-only optimization considers the supported feasibility conditions without the same carbon objective. Carbon-aware allocation incorporates intensity-related costs while respecting the modeled constraints. The unconstrained arm helps show the consequences of removing restrictions; it must not be described as a deployable policy if it ignores requirements that users actually need.

The optimization uses OR-Tools CP-SAT. Constraint programming represents admissible decisions and relationships explicitly, while the solver searches for an assignment meeting those conditions and optimizing an objective. Rolling allocation repeatedly solves using the currently considered horizon. Its quality and computation time depend on workload size, time discretization, supply pressure, and the structure of the constraints.

The replay rescales arrival gaps into a bounded experimental episode. Some durations and power values are modeled, and the supply scenario is configured. Historical intensity scores the resulting schedule. These choices make experiments tractable and repeatable, but they mean the result is a counterfactual replay

rather than an observed deployment outcome. Changing scarcity or deferral allowances can change the apparent benefit of a scheduling strategy.

Completions, overruns, energy, modeled emissions, cost, and waiting time should be read together. A scheduler that reduces emissions by dropping more jobs is not automatically better. Likewise, delaying tasks can move them into cleaner intervals while harming latency. Paired seeds and confidence intervals help quantify uncertainty, but they do not remove bias in workload selection or modeling assumptions.

Pricing agents and finite state explanations

The pricing experiments include cost-plus, scarcity, scripted-response, and Q-learning strategies. Q-learning is a reinforcement learning method that estimates the value of actions in states using rewards and subsequent observations. Here, the implementation trains tabular agents in repeated inventory-market episodes. A Q-table exposes the learned action values directly, making the behavior easier to inspect than an opaque large model.

Policies are frozen for evaluation. Distinct episode sets are used to fit a probabilistic finite state representation, calibrate thresholds, and test behavior. A finite state machine summarizes behavior using states and transitions. Its predictive distribution can be evaluated by asking how well it anticipates the next action. State coverage indicates whether held-out observations map into the learned representation; it does not mean every possible real market condition has been observed.

The intervention detector compares responses to a genuine controlled price cut with responses to a sham intervention. This is stronger than declaring every high price suspicious. Scarcity or a fixed premium can also produce high prices without the configured response pattern. The controlled benchmark includes those negative controls so that the detector can be evaluated on its ability to distinguish the intended behavior.

The auditor flags observable pricing patterns for review. It does not establish legal collusion, intent, or wrongdoing by real providers. The positive labels in the experiment come from configured synthetic behavior. Unlabelled trajectories from learned agents are not silently converted into ground-truth collusion labels. This boundary should be stated explicitly when presenting an apparently perfect F1 score.

Outcomes and interpretation of metrics

The extraction regression contains 180 template examples: 120 development, 30 validation, and 30 holdout cases. The fresh review reproduced exact expected outputs for all 180 examples. This is 100 percent exact match within the supported template corpus. It is not 100 percent accuracy on arbitrary legal agreements. Independent human annotation and out-of-template contracts are needed to evaluate generalization.

The protocol study reports 72 held-out trace cases across six specified families. The temporal monitor matches 72 of 72 expected verdicts and accepts zero unsafe traces. The event-presence baseline matches 24 of 72 and accepts 48 unsafe traces. Verdict accuracy is correct verdicts divided by the number of cases. Unsafe acceptance counts matter separately because two monitors with similar accuracy can have very different safety consequences.

The controlled pricing detector reports F1 of 1.000 on 120 labelled held-out episodes, consisting of 30 response positives and 90 controls. The high-markup baseline reports F1 of 0.667 with 30 false positives. Precision measures how many flagged positives are correct; recall measures how many actual positives are detected. F1 is their harmonic mean. These denominators and synthetic labels are essential context for the perfect result.

For action prediction, the merged 13-state FSM reports 79.31 percent accuracy and log loss of 0.548 over 15,750 held-out action observations across 450 trajectories. The majority-action baseline reaches 74.94

percent accuracy and log loss of 0.737. Lower log loss indicates better probability assignment to observed actions. Accuracy alone ignores whether a model was mildly or extremely confident in an incorrect prediction.

The calibrated runtime bound covers all 46 held-out successful lifecycle proxies, but this apparently strong coverage comes at a cost. Its p90 pinball loss is approximately 6,934 seconds, compared with 2,221 seconds for the training-p90 baseline, and its average bound is approximately 74,668 seconds. A very wide bound can cover many observations while reserving impractically long intervals. This result is evidence of overconservatism, not a production-ready runtime predictor.

The 20-seed publication replay uses 90 jobs per seed. With calibrated durations, carbon-aware scheduling averages 83.60 completions, 5.60 overruns, and approximately 1,846 grams of modeled CO2. With median durations, it averages 83.85 completions, 5.75 overruns, and approximately 1,855 grams. The calibrated-minus-median carbon difference is about minus 9.4 grams per run, with a 95 percent bootstrap interval of approximately minus 29.1 to plus 9.1 grams. The interval crosses zero, so there is no clear calibration benefit.

Within the calibrated replay, carbon-aware allocation averages approximately 1,845.7 grams compared with 2,014.6 grams for policy-only allocation. Its p95 wait is 48 minutes versus 21 minutes, while average completions are 83.60 versus 83.65. The approximately 169-gram modeled reduction therefore accompanies approximately 27 additional minutes of p95 waiting. Reporting the carbon number without these service outcomes would conceal the practical trade-off.

Explainable interface and operational support

Explainable AI provides role-specific journeys, clickable execution traces, trained Q-tables, pricing interventions, finite state transitions, runtime comparisons, and downloadable research material. These views connect a conclusion to its underlying evidence. The purpose is to help a user answer why a job was rejected, how a booking progressed, or what a research result actually measures.

The interface also contains reviewer and incident workflows, counterfactual previews, lists with pagination, and event-driven updates. Accessibility work includes mobile navigation focus handling, Escape support, focus return, and alternatives for chart information. Automated accessibility checks provide useful regression coverage, but they do not replace screen-reader testing, manual keyboard inspection, or human assessment of graph comprehension.

The support benchmark uses 50 developer-authored cases. Passing those cases establishes conformance to the encoded expectations. It does not establish that operators independently found the guidance sufficient or that a live language model never produces an inappropriate response. Optional model assistance should be described separately from the deterministic guidance and evidence inspection that work without an external provider.

Running deployment and maintenance

The faculty launcher supplied with this review starts a separate local installation at port 8767. It prepares local demo accounts, imports source data, creates research results when absent, prepares the model, and launches the API, worker, and runner. Its state lives under .local/faculty-demo. The pre-existing installation at port 8765 and its configured PostgreSQL database remain separate. This separation makes rehearsal repeatable without changing the team's shared records.

On this laptop, the existing Python 3.14 application environment loads the native research libraries. A separate Python 3.12 environment supplies the prepared GPU runtime. Newly installed OR-Tools and Parquet libraries in the clean Python 3.12 environment are blocked by Windows Application Control. The

launcher uses the working application environment and does not disable that operating-system protection. The environment arrangement should be recorded when transferring the project to another machine.

Hosting uses the repository's Dockerfile to build the frontend and package the Python API and worker. PostgreSQL, an HTTPS origin, stable signing credentials, and durable artifact storage must be configured. A separate GPU host points its runner at that HTTPS origin. The deployment procedure must preserve secrets, back up database and artifacts together, verify migration compatibility, and test an actual end-to-end hosted job. A local container volume declaration alone does not make a provider's ephemeral disk durable.

The existing Render endpoint returned HTTP 200 with PostgreSQL health after a cold-start retry during this review. This establishes reachability and database health at that time. The deployed commit, storage durability, worker state, and remote GPU path still require deployment-specific verification. Render documents that free web services can sleep and lose local files across restarts, so a successful page load is insufficient evidence of reliable artifact retention.

Fresh laptop validation outcomes

The review on this RTX 4050 laptop passed 265 backend regression tests with six skips. Five skips concern the separate disposable PostgreSQL suite; the remaining skip is an authorization scenario covered by another test. The frontend passed 33 unit tests and the TypeScript production build. Four browser workflows passed with no failed or flaky cases. These browser workflows exercised a real GPU job, rejected memory placement, research replay, marketplace contention and isolation, role invitations, reviewer approval, and responsive interface behavior.

The successful browser workload produced four embeddings of dimension 960. Its artifact hash was checked against the bytes downloaded from the API. GPU inference took 0.06491 seconds, while reported GPU board energy was 0.00001878 kWh with telemetry coverage of 0.912, or 91.2 percent. Inference time is not total request latency; board energy is not whole-system energy. Preparation reported CPU-reference cosine similarity of 1.0 and a maximum numerical difference of approximately 0.000000522. These checks validate the tested implementation and input, not general image classification accuracy.

The review found and corrected missing development dependencies and an incompatible optional monitoring package. The existing application environment now passes pip dependency consistency checks. Frontend dependencies were synchronized with the lockfile, whose install-time audit reported zero known findings. This narrow audit result is not an independent security assessment. The clean Python 3.12 application environment remains blocked by Windows native-library policy, while the tested existing application environment and separate GPU runtime provide a working local demonstration.

Future improvements

The most valuable research improvement is independent evaluation: human-labelled agreements, partner workload traces, realistic failures, and measurements from more than one authorized GPU host. Runtime estimation needs better targets and calibration before integration into live reservations. Pricing detection needs broader controls and independently justified labels. Scheduling should be evaluated under heavier load with fairness, cost, waiting time, and completion outcomes retained alongside carbon estimates.

Operational improvements include reproducible deployment identities, durable storage checks, backup restoration drills, monitoring of hung workers, stronger identity integration such as OIDC, and explicit service objectives. Broader workloads and multiple GPUs require a more general isolation and scheduling design. Paid billing and marketplace settlement would require a separate accounting implementation. Formal synthesis, external automata-learning integration, and commercial collusion determination should remain future work until they are implemented and independently evaluated.

Evidence and reference locations

Implementation evidence is in backend/pact, frontend/src, backend/migrations, and the automated tests. Fresh command logs and review results are stored in reports/user-review. Research evidence is generated under .local/validation/existing-research/evaluations and the faculty demo's evaluations directory. Historical validation remains under reports/final-validation and reports/release-candidate and should not be confused with fresh results on this laptop.

Primary data sources are the Alibaba GPU trace repository at https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2023 and the NESO carbon intensity API at https://api.carbonintensity.org.uk/. Platform behavior is documented at https://render.com/docs/free. The repository's RESEARCH_REVIEW.md records method references and corrects an earlier mismatched literature attribution. Use the implemented behavior and measured evidence when answering questions, rather than treating every item in an older roadmap as a completed feature.